

3. Работаем с SQL

- Небольшая практика в этом языке
 - Польза и вред ограничений
 - Пробуем индексы

```
SELECT airport_code, counted
FROM (
  SELECT airports_data.*, COUNT(airports_data.airport_code) as counted
  FROM airports_data INNER JOIN flights
  ON (airports_data.airport_code = flights.departure_airport OR airports_data.airport_code
= flights.arrival_airport)
  GROUP BY airports_data.airport_code
) as subquery
ORDER BY counted DESC
LIMIT 5
```

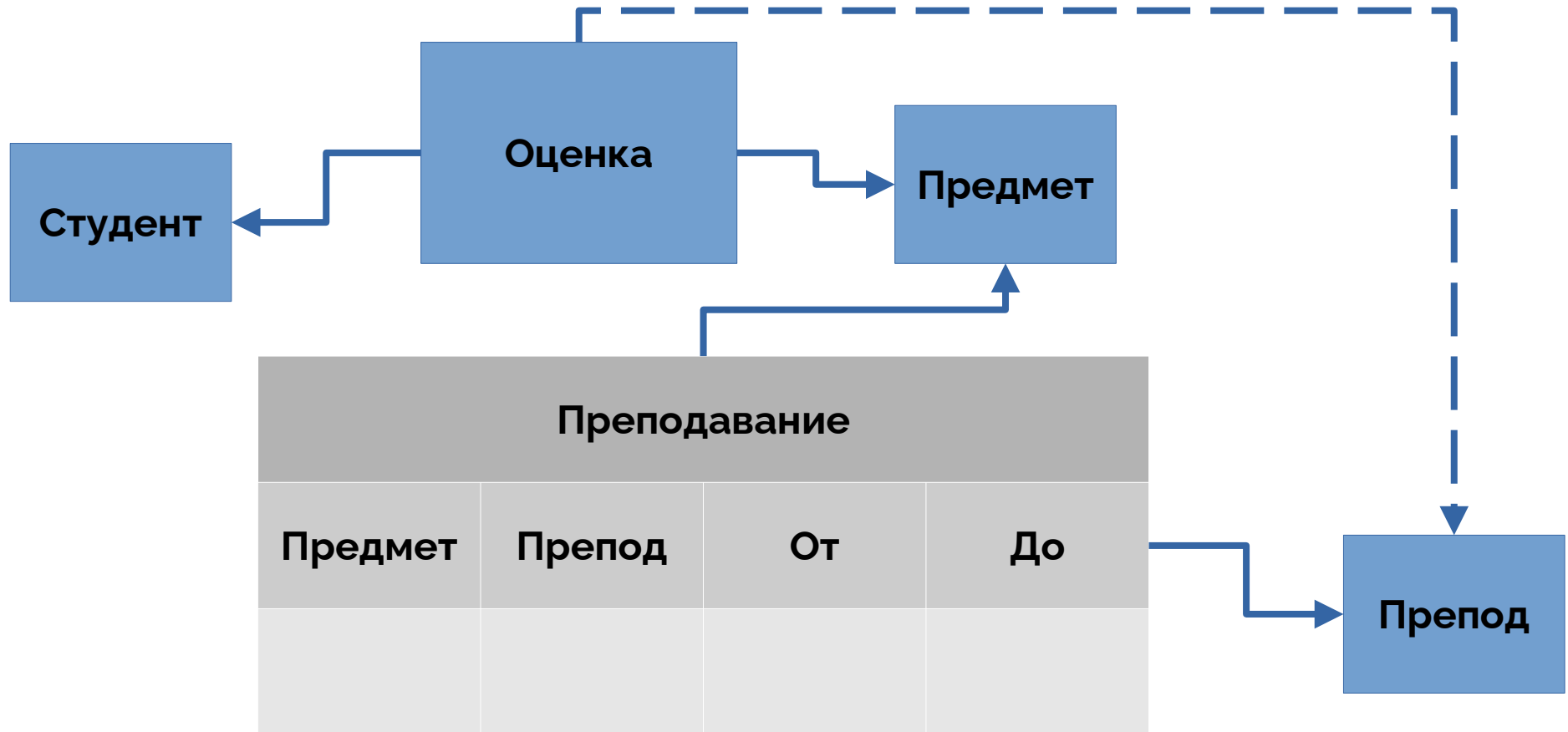
```
WITH airports_by_boarding_passes AS (  
    SELECT departure_airport, arrival_airport  
    FROM bookings.boarding_passes bp JOIN bookings.flights f ON bp.flight_id = f.flight_id  
),  
all_airports AS (  
    SELECT departure_airport as airport  
    FROM airports_by_boarding_passes  
    UNION ALL  
    SELECT arrival_airport as airport  
    FROM airports_by_boarding_passes  
)  
SELECT airport, count(airport) cnt FROM all_airports GROUP BY airport ORDER BY cnt  
DESC LIMIT 5;
```

```
WITH airport_visits AS (  
    SELECT departure_airport AS airport, COUNT(*) AS passengers_cnt  
    FROM Ticket_flights  
    JOIN flights ON Ticket_flights.flight_id = flights.flight_id  
    JOIN Boarding_passes ON Ticket_flights.ticket_no = Boarding_passes.ticket_no  
    AND Ticket_flights.flight_id = Boarding_passes.flight_id  
    GROUP BY departure_airport  
)  
SELECT * from airport_visits  
ORDER BY passengers_cnt DESC  
LIMIT 5;
```

```
SELECT ap.airport_code, ap.airport_name,  
       COUNT(CASE WHEN f.departure_airport = ap.airport_code THEN 1 END) AS  
departures_count,  
       COUNT(CASE WHEN f.arrival_airport = ap.airport_code THEN 1 END) AS  
arrivals_count,  
       COUNT(CASE WHEN f.departure_airport = ap.airport_code THEN 1 END) +  
       COUNT(CASE WHEN f.arrival_airport = ap.airport_code THEN 1 END) AS  
total_passengers  
FROM flights f  
JOIN airports ap ON f.departure_airport = ap.airport_code OR f.arrival_airport =  
ap.airport_code  
GROUP BY ap.airport_code, ap.airport_name  
ORDER BY total_passengers DESC  
LIMIT 5;
```

```
SELECT
    ad.airport_code,
    ad.airport_name->>'ru' AS airport_name,
    COUNT(*) AS passenger_count
FROM
    bookings.ticket_flights tf
JOIN
    bookings.flights f ON tf.flight_id = f.flight_id
JOIN
    bookings.airports_data ad ON ad.airport_code = ANY
    (ARRAY[f.departure_airport, f.arrival_airport])
GROUP BY
    ad.airport_code, ad.airport_name
ORDER BY
    passenger_count DESC
LIMIT 5;
```

«Реальность сложнее»



Запустим постгрес

```
psql -h host -p port -U user dbname
```

Host : адрес или путь к каталогу с сокетом в ФС

Port : порт TCP или условный порт (имя сокета)

```
sudo -u postgres psql
```


Что там внутри?

`\d`

`\l`

`\c db_name [ user_name ]`

`\help SELECT`

`CREATE USER`

`CREATE DATABASE`

SQL

```
CREATE TABLE имя ( список полей );
```

SQL

```
CREATE TABLE student (...)  
CREATE TABLE subject (...)  
CREATE TABLE teacher (...)  
CREATE TABLE teaching (...)  
CREATE TABLE exam (...)
```

SQL

```
CREATE SEQUENCE id_seq START WITH 10000000;  
CREATE TABLE student (  
    id          int          PRIMARY KEY DEFAULT nextval('id_seq'),  
    name        text  
);  
CREATE TABLE subject ()  
CREATE TABLE teacher ()  
CREATE TABLE teaching(  
    id          int          PRIMARY KEY DEFAULT nextval('id_seq'),  
    subject     int          NOT NULL REFERENCES subject(id),  
    teacher     int          NOT NULL REFERENCES teacher(id),  
    period      tstzrange NOT NULL  
);  
CREATE TABLE exam (...)
```

CHECK CONSTRAINT

(ограничение-проверка)

```
CREATE TABLE exam (  
    id            int    PRIMARY KEY DEFAULT nextval ('id_seq'),  
    subject       int    NOT NULL REFERENCES subject(id),  
    teacher       int    NOT NULL REFERENCES teacher(id),  
    student       int    NOT NULL REFERENCES student(id),  
    score         int    NOT NULL  
  
); -- оценка должна быть  
   -- от 1 до 10
```



CHECK CONSTRAINT

(ограничение-проверка)

```
CREATE TABLE exam (  
    id          int          PRIMARY KEY DEFAULT  
nextval('id_seq'),  
    subject     int          NOT NULL REFERENCES subject(id),  
    teacher     int          NOT NULL REFERENCES teacher(id),  
    student     int          NOT NULL REFERENCES student(id),  
    score       int          NOT NULL,  
  
[ CONSTRAINT score_range ]  
    CHECK (score > 0 AND score <=10)  
  
);
```

DOMAIN

(другой способ)



DOMAIN

(другой способ)

```
CREATE DOMAIN exam_score AS int
    NOT NULL
    CHECK (VALUE > 0 AND VALUE <=10);
```

```
CREATE TABLE exam (
    id          int          PRIMARY KEY DEFAULT
nextval('id_seq'),
    subject     int          NOT NULL REFERENCES subject(id),
    teacher     int          NOT NULL REFERENCES teacher(id),
    student     int          NOT NULL REFERENCES student(id),
    score       exam_score
);
```


А вот такой CHECK CONSTRAINT:

```
ALTER TABLE teacher  
  ADD inn text  
  CHECK (length(inn)=10 OR length(inn)=12);
```

Хорошо это или худо?

UNIQUE CONSTRAINT

(ограничение уникальности)

```
CREATE TABLE exam (  
    id            int    PRIMARY KEY DEFAULT nextval ('id_seq'),  
    subject       int    NOT NULL REFERENCES subject(id),  
    teacher       int    NOT NULL REFERENCES teacher(id),  
    student       int    NOT NULL REFERENCES student(id),  
    score         int    NOT NULL,  
  
    UNIQUE(subject, student)  
  
);
```

\d exam

Два преподавателя не могут
сидеть в одной реке



Варианты толкования:

Структурное

«Меня тут не может быть двое»!

teaching:
 teacher
 subject
 period -- периоды не должны пересекаться

Прикладное

«Нас тут не может быть двое»!

teaching:
 subject
 period -- периоды не должны пересекаться

Если надо однозначно дать ответ на вопрос, кто преподавал этот предмет в этот момент времени

EXCLUSION CONSTRAINT

Ограничения - исключения



EXCLUSION CONSTRAINT

Ограничения - исключения

```
CREATE EXTENSION btree_gist;

CREATE TABLE teaching (

    id            int            PRIMARY KEY,
    subject       int            NOT NULL REFERENCES subject(id),
    teacher       int            NOT NULL REFERENCES teacher(id),
    period        tstzrange NOT NULL,

    EXCLUDE USING gist (
        subject WITH =,
        teacher WITH =,
        period WITH && )

);
```

EXCLUSION CONSTRAINT

Добавление ограничения

```
ALTER TABLE teaching ADD CONSTRAINT no_dup  
    EXCLUDE USING gist (  
        subject WITH =,  
        teacher WITH =,  
        period WITH &&  
  
    );
```

Возник индекс

```
\d teaching
```

```
"no_dup" EXCLUDE USING gist (subject WITH =,  
teacher WITH =, period WITH &&)
```


Вопросы

Что это мы сейчас сделали?

1) **Индекс**

- специальная структура для избегания полного перебора при поиске
- внутреннее устройство м.б. разным (в силу расширяемости Postgres)

2) **GiST (Generalized Index Search Tree)**

- тип индекса (он же «метод доступа»).
- разработан О. Бартуновым и Ф. Сигаевым в 2000г для поиска по вхождению в массив (для индекса RD-tree)
- позже на его базе реализованы другие типы индексов (в частности, B-Tree)

3) Расширение **btree_gist** — как раз для этого. Чтобы объединить int и tstzrange в одном индексе.

Заполним таблицы

Как нам сгенерировать тестовые данные?

Полезные функции: `generate_series`, `random`

Другое полезное:

`substring(text, int, int)`

`string_agg`

конкатенация строк: `||`

Нет функции, которая генерирует случайную строку.

Давайте ещё напишем...

КОНЕЦ

К следующему разу:

Создать у себя учебную БД «Экзамены и студенты», обсуждавшуюся на лекции.
Придумать, как генерировать случайные строки.
С помощью этого заполнить БД тестовыми данными.

Что будем делать:

посмотрим на транзакции и блокировки
детальнее поиграемся в индексы
будем анализировать БД «Авиаперевозки»
ещё посмотрим на планы запросов